Practical 5: HTCondor Job Submission Practical Lab

In the Practical 4, students learned the architecture of HTCondor and follow the instruction to set up a 4-nodes Condor Pool.

This lab introduces High Throughput Computing (HTC) principles using HTCondor to manage, schedule, and execute different distributed workloads across compute clusters. Students explore how distributed workload managers distribute tasks, handle file transfer mechanisms, manage environment execution dependencies, and enforce node hardware requirements.

Types of job submitted to HTCondor

HTCondor is designed to support various type of jobs with the execution universe (refer to the HTCondor documentation. There are THREE commonly used in this course:

Aspect / Directive	Bash Shell Script	Java Application (Vanilla Universe)	Compiled C Binary
HTCondor Universe (universe)	vanilla	vanilla	vanilla
Executable Path (executable)	Path to shell script (e.g., batch_job.sh)	System path to host JVM executable (e.g., /usr/bin/java)	Path to compiled binary file (e.g., sys_inquiry)
Arguments Syntax (arguments)	Script command-line parameters (e.g., $(Process))	Main class name followed by application parameters (e.g., DataProcessor input.txt results.txt)	Standard CLI arguments passed to main(int argc, char *argv[])
Prerequisites & Build Step	Text script with shell shebang (#!/bin/bash) and execute permissions (chmod +x)	Pre-compiled bytecode generated via javac DataProcessor.java	Source code compiled to machine code for target architecture (e.g., gcc sys_inquiry.c -o sys_inquiry)
File Transfer Configuration (transfer_input_files)	List supporting input data files or secondary scripts	Must explicitly list all .class files alongside input data files	List input data files (executable binary transfers automatically)
Worker Node Requirements	Matching shell interpreter available on worker host	Target worker node must have JVM installed at the specified absolute path	Binary architecture and OS kernel compatibility
Primary Use Case	Automation scripts, pipeline wrappers, and simple workflow orchestration	Java applications where HTCondor Java universe is unconfigured or custom JVM flags are needed	High-performance CPU-intensive workloads, system hardware queries, or native binaries

Question 1 — Simple Batch Script Job

Create a Bash shell script named batch_job.sh and submit it as a multi-process job array to HTCondor.

1. Write batch_job.sh to print the execution host's hostname, current date/time, job process ID passed as a command-line argument, and simulate a work delay of 5 seconds.
2. Create an HTCondor submit description file batch_job.sub that:
    - Sets the universe to vanilla.
    - Passes the HTCondor process ID ($(Process)) as an argument.
    - Redirects standard output, standard error, and job log to distinct files named job_<Process_ID>.out, job_<Process_ID>.err, and batch_job.log.
    - Queues 3 instances of the job.
3. Submit the job, monitor queue status, and verify the output files.

Sample Output:

```
ubuntu@ip-172-31-20-133:~/example01$ cat job_0.out
========================================
Starting Job Process ID: 0
Running on Host: ip-172-31-30-237
Start Time: Mon Nov  2 06:08:56 UTC 2026
========================================
```

Question 2 — Java Job with File I/O and Processing

Build a Java application that processes numerical data from an input file and submits it for execution on an HTCondor worker node.

1. Write a Java program DataProcessor.java that:
    - Accepts input filename and output filename via command-line arguments (args[0] and args[1]).
    - Reads a list of floating-point numbers from the input file.
    - Calculates total count, sum, arithmetic mean, minimum, and maximum values.
    - Writes the formatted summary to the specified output file.
2. Compile DataProcessor.java into bytecode (DataProcessor.class).
3. Prepare a test data file input.txt containing at least 5 numbers.
4. Write an HTCondor submit file java_job.sub using universe = vanilla (running Java via the local JVM executable /usr/bin/java) that explicitly transfers DataProcessor.class and input.txt to the execute node and returns results.txt.

Sample Input:

```
53
4
10
7
```

Sample Output:

```
=== HTCondor Data Processing Summary ===
Total Records Processed: 4
Sum:                    74.00
Average:                18.50
Minimum Value:          4.00
Maximum Value:          53.00
========================================
```

Question 3 — Compiled C Job for System Hardware & OS Inquiry

Write, compile, and execute a C program on HTCondor to query hardware specifications and system properties of assigned execution nodes.

1. Write a POSIX-compliant C program sys_inquiry.c using standard system headers (<sys/utsname.h> and <unistd.h>) to gather and print:
    a. Node hostname (nodename)
    b. Operating System kernel name and release version
    c. Machine architecture (e.g., x86_64)
    d. Current process ID (PID) assigned by the host OS
    e. Number of active CPU cores (_SC_NPROCESSORS_ONLN)
    f. Total physical RAM installed in Megabytes (_SC_PHYS_PAGES * _SC_PAGESIZE)
2. Compile the C source code into a binary executable named sys_inquiry.
3. Create an HTCondor submit file c_job.sub to submit the compiled binary.

Sample output:

```
================================================
        HTCondor Node Hardware & OS Report
================================================
Host Name:         ip-172-31-23-219
OS Kernel:         Linux
OS Release:        7.0.0-1014-aws
Architecture:      x86_64
Assigned Process ID: 949
Available CPU Cores: 2
Total Physical RAM:  1905 MB
================================================
```